

Project Update Report

Course: CSE440 (Section 01)

Project: Console-Based Tic-Tac-Toe Game

Role focus: User interaction (menu.py, player.py)

Week 1 (Jun 10 – Jun 16)

I focused on how the player interacts with the game through terminal input. Since this is a console project, good prompts and error messages are very important. If input handling is weak, the game feels broken even when logic is correct. I listed possible user mistakes: entering letters, typing 0 or 10, selecting an already filled cell, or choosing invalid menu options. I also thought about less obvious cases, such as extra spaces, pressing enter with nothing typed, or a decimal like 3.5 instead of an integer.

Then I planned validation rules, so the program guides the user instead of crashing. I sketched a rough flow: read input, check if it is numeric, check if it is in range, check if the cell is free, and only then accept the move. If any step failed, the plan was to print a specific message and ask again rather than exiting or throwing a raw error. This planning stage took longer than expected, but it made the actual coding in Week 2 much faster since I already knew what each function needed to check.

Week 2 (Jun 17 – Jun 23)

I implemented two modules: `support/menu.py` displays title and mode options [1] and [2], keeps asking until input is valid. `support/player.py` reads move input for human players, validates numeric range (1-9), checks occupied cells, lets the same player retry without losing turn. This retry behavior was important for fairness, especially in two-player mode. We did not want one typo to skip someone's turn.

While building `menu.py`, I made sure the menu keeps looping instead of exiting on the first bad entry. I used simple string checks combined with `try/except` blocks for numeric conversion, so that letters or symbols typed by mistake are caught cleanly instead of crashing the program with a traceback.

For `player.py`, I separated the validation into small steps: first confirm the input can be converted to an integer, then confirm it falls between 1 and 9, and finally confirm the chosen cell is not already occupied on the board. Splitting it this way made debugging easier, since I could test each condition on its own instead of guessing which part of a combined check was failing.

Challenges Faced

I ran into a few real problems while writing and correcting these functions. The first was an off-by-one error in the range check: I had written `1 <= move < 9` instead of `1 <= move <= 9`, which silently rejected cell 9 even though it is a legal position. It took a while to catch because the error message printed was the generic “Out of bounds” text, so I first assumed the user was typing something wrong rather than the check itself being wrong.

The second issue was a type mismatch: the board stored positions as strings while `player.py` converted input to an integer, so `if move in occupied_cells` was always `False` even when a cell was taken. This let players overwrite each other's marks. I fixed it by converting the move back to a string before checking membership.

The retry logic also caused a subtler bug. My first `while` loop used a `continue` statement to re-prompt on invalid input, but I forgot to reset a flag inside the loop, which occasionally made the function loop forever on a specific bad-input pattern (an empty string followed by a letter). I added a temporary print statement

to trace the loop count before noticing the flag was never reset, and fixed it by resetting the flag at the top of the loop body.

On top of that, empty input crashed the numeric conversion with an uncaught `ValueError`, since I had only anticipated letters, not blank strings, and extra whitespace like " 3 " failed the numeric check until I added a `.strip()` call before conversion. None of these felt like big problems individually, but together they took up most of Week 2.

Week 3 (Jun 24 – Jun 30)

I tested interaction quality manually by running many input scenarios: invalid menu values, repeated occupied positions, and mixed valid/invalid move patterns. The prompts now give clear feedback like “Invalid choice,” “Out of bounds,” and “That spot is already taken.” I also reviewed message consistency between one-player and two-player modes (for example, “You (X)” vs “Player 1 (X)”).

One testing session turned up a bug that had slipped through Week 2: entering an occupied cell followed by a valid move incremented the turn counter twice instead of once. Tracing it back, the retry function was returning control to the main loop even on a rejected input, so the loop advanced the turn on both the invalid attempt and the valid one that followed. I fixed it by moving the turn-increment logic so it only runs after `player.py` returns a confirmed valid move.

Status: menu and input validation modules are complete and connected with the main game loop.

Summary

Overall, this period was about making the console interface feel reliable rather than fragile. Planning the validation rules before coding in Week 1 saved time later, and the bugs caught in Week 2 and Week 3 made the input layer noticeably more solid. With `menu.py` and `player.py` finished and integrated with the main loop, the next step is to coordinate with teammates on connecting these modules with the board display and win-checking logic.